

fbfrontend - Phase 8.4

Shared WebSocket lifecycle

Complete source files and before/after changes

8 files: 4 updated and 4 added. Implemented locally on top of Phase 8.3.

Three small adapters use one shared STOMP lifecycle. Physical connections remain independent. Destinations, callback names and REST-confirmed UI sends are retained.

Pending connections stop after logout/disposal. Stale refresh responses cannot restore logged-out tokens or overwrite/clear a newer login. Authentication errors have a controlled reconnect path.

Verified: 25 unit tests, existing chat and sidebar browser regressions, and new live-auth browser checks. Mocked local services only; no cloud data changes.

Production build passed. Gzip JavaScript decreased from 114.84 to 114.09 kB. CSS unchanged. No lint errors; same 13 existing warnings.

Backend and FrndBookWC2 unchanged. No new dependencies, migration or deployment. The existing refresh-failure policy and backend authorization remain in force.

Full current source and unified diffs use Phase 8.3 as the baseline. Green + lines are added; red - lines are removed. Gutter numbers are aids; long lines wrap with >.

The companion Plan and Testing Markdown contains design decisions, limitations, commands and manual acceptance checks.

File index

src/api/axiosClient.js	3
src/services/authenticatedWebSocket.js	7
src/services/chatWebSocketService.js	9
src/services/conversationUpdateWebSocketService.js	15
src/services/stompConnection.js	21
src/services/webSocketService.js	25
tests/liveAuth.browser.mjs	31
tests/stompConnection.test.js	35

```

1  import axios from "axios";
2
3  import { tokenStorage } from "../utils/tokenStorage";
4
5  const axiosClient = axios.create({
6    baseURL: import.meta.env.VITE_API_BASE_URL,
7
8    // ISSUE FIX:
9    // Do not set Content-Type globally.
10   // Profile image uploads use FormData and must be sent as
11   // multipart/form-data with the boundary generated automatically.
12 });
13
14 // =====
15 // REFRESH STATE
16 // =====
17
18 let refreshPromise = null;
19
20 // =====
21 // REFRESH ACCESS TOKEN
22 // =====
23
24 export const refreshAccessToken = async () => {
25   /*
26    * If another refresh is already running,
27    * everyone waits for that same request.
28    */
29   if (refreshPromise) {
30     return refreshPromise;
31   }
32
33   const refreshToken = tokenStorage.getRefreshToken();
34
35   if (!refreshToken) {
36     throw new Error("Refresh token is not available");
37   }
38
39   refreshPromise = axios
40     .post(`${import.meta.env.VITE_API_BASE_URL}/api/auth/refresh`, {
41       refreshToken,
42     })
43     .then((response) => {
44       if (tokenStorage.getRefreshToken() !== refreshToken) {
45         throw new Error("Authentication changed while refreshing");
46       }
47       const { accessToken, refreshToken: newRefreshToken } = response.data;
48
49       tokenStorage.updateTokens(accessToken, newRefreshToken);
50
51       return accessToken;
52     })
53     .catch((error) => {
54       if (tokenStorage.getRefreshToken() === refreshToken) tokenStorage.clear();
55
56       throw error;
57     })
58     .finally(() => {
59       refreshPromise = null;
60     });
61
62   return refreshPromise;
63 };
64
65 // =====
66 // REQUEST INTERCEPTOR
67 // =====
68
69 axiosClient.interceptors.request.use(
70   (config) => {

```

```

71     const accessToken = tokenStorage.getAccessToken();
72
73     if (accessToken) {
74         config.headers.Authorization = `Bearer ${accessToken}`;
75     }
76
77     return config;
78 },
79
80 (error) => {
81     return Promise.reject(error);
82 },
83 );
84
85 // =====
86 // RESPONSE INTERCEPTOR
87 // =====
88
89 axiosClient.interceptors.response.use(
90     (response) => {
91         return response;
92     },
93
94     async (error) => {
95         const originalRequest = error.config;
96
97         // -----
98         // Only handle 401
99         // -----
100
101         if (error.response?.status !== 401) {
102             return Promise.reject(error);
103         }
104
105         // -----
106         // Prevent infinite retry
107         // -----
108
109         if (originalRequest?._retry) {
110             return Promise.reject(error);
111         }
112
113         // -----
114         // Never refresh these endpoints
115         // -----
116
117         const requestUrl = originalRequest?.url || "";
118
119         /*
120          * ISSUE FIX:
121          * All public authentication endpoints must bypass
122          * the access-token refresh flow.
123          */
124         if (
125             requestUrl.includes("/api/auth/login") ||
126             requestUrl.includes("/api/auth/signup") ||
127             requestUrl.includes("/api/auth/verify-email") ||
128             requestUrl.includes("/api/auth/resend-verification") ||
129             requestUrl.includes("/api/auth/refresh") ||
130             requestUrl.includes("/api/auth/forgot-password") ||
131             requestUrl.includes("/api/auth/reset-password")
132         ) {
133             return Promise.reject(error);
134         }
135
136         // -----
137         // Mark request
138         // -----
139
140         originalRequest._retry = true;

```

```
141
142   try {
143     const newAccessToken = await refreshAccessToken();
144
145     originalRequest.headers.Authorization = `Bearer ${newAccessToken}`;
146
147     return axiosClient(originalRequest);
148   } catch (refreshError) {
149     /*
150      * Refresh failed completely.
151      * The shared refresh function has already
152      * cleared authentication.
153      */
154
155     window.location.href = "/login";
156
157     return Promise.reject(refreshError);
158   }
159 },
160 );
161
162 export default axiosClient;
```

src/api/axiosClient.js

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- before/src/api/axiosClient.js
2 +++ after/src/api/axiosClient.js
3 @@ -41,6 +41,9 @@
4     refreshToken,
5   })
6   .then((response) => {
7 +   if (tokenStorage.getRefreshToken() !== refreshToken) {
8 +     throw new Error("Authentication changed while refreshing");
9 +   }
10     const { accessToken, refreshToken: newRefreshToken } = response.data;
11
12     tokenStorage.updateTokens(accessToken, newRefreshToken);
13 @@ -48,7 +51,7 @@
14     return accessToken;
15   })
16   .catch((error) => {
17 -   tokenStorage.clear();
18 +   if (tokenStorage.getRefreshToken() === refreshToken) tokenStorage.clear();
19
20     throw error;
21   })
```

src/services/authenticatedWebSocket.js

FULL CURRENT CONTENT

```
1 import { Client } from "@stomp/stompjs";
2 import { refreshAccessToken } from "../api/axiosClient";
3 import { tokenStorage, AUTH_CLEARED_EVENT } from "../utils/tokenStorage";
4 import { createStompConnection } from "./stompConnection.js";
5
6 export const createAuthenticatedWebSocket = options => createStompConnection({
7   ...options,
8   createClient: config => new Client(config),
9   brokerURL: import.meta.env.VITE_WS_URL,
10  getToken: () => tokenStorage.getAccessToken(),
11  refreshToken: refreshAccessToken,
12  authEvents: window,
13  authClearedEvent: AUTH_CLEARED_EVENT,
14 });
```

src/services/authenticatedWebSocket.js

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- /dev/null
2 +++ after/src/services/authenticatedWebSocket.js
3 @@ -0,0 +1,14 @@
4 +import { Client } from "@stomp/stompjs";
5 +import { refreshAccessToken } from "../api/axiosClient";
6 +import { tokenStorage, AUTH_CLEARED_EVENT } from "../utils/tokenStorage";
7 +import { createStompConnection } from "../stompConnection.js";
8 +
9 +export const createAuthenticatedWebSocket = options => createStompConnection({
10 +  ...options,
11 +  createClient: config => new Client(config),
12 +  brokerURL: import.meta.env.VITE_WS_URL,
13 +  getToken: () => tokenStorage.getAccessToken(),
14 +  refreshToken: refreshAccessToken,
15 +  authEvents: window,
16 +  authClearedEvent: AUTH_CLEARED_EVENT,
17 +});
```

src/services/chatWebSocketService.js

FULL CURRENT CONTENT

```
1 import { createAuthenticatedWebSocket } from "../authenticatedWebSocket";
2
3 export const createChatWebSocket = ({ conversationId, ...callbacks }) => {
4   if (!conversationId) return null;
5   return createAuthenticatedWebSocket({
6     ...callbacks,
7     destination: `/topic/conversations/${conversationId}`,
8     publishDestination: `/app/chat/${conversationId}`,
9   });
10  };
```

```

1  --- before/src/services/chatWebSocketService.js
2  +++ after/src/services/chatWebSocketService.js
3  @@ -1,306 +1,10 @@
4  -import { Client } from "@stomp/stompjs";
5  +import { createAuthenticatedWebSocket } from "../authenticatedWebSocket";
6
7  -import { refreshAccessToken } from "../api/axiosClient";
8  -
9  -import { tokenStorage, AUTH_CLEARED_EVENT } from "../utils/tokenStorage";
10 -
11 -const WS_URL = import.meta.env.VITE_WS_URL;
12 -
13 -const RECONNECT_DELAY = 5000;
14 -
15 -const TOKEN_REFRESH_BUFFER_SECONDS = 30;
16 -
17 -// =====
18 -// JWT EXPIRATION
19 -// =====
20 -
21 -const getTokenExpiration = (token) => {
22 -  try {
23 -    const parts = token.split(".");
24 -
25 -    if (parts.length !== 3) {
26 -      return null;
27 -    }
28 -
29 -    const payload = JSON.parse(
30 -      atob(parts[1].replace(/-/g, "+").replace(/_/g, "/")),
31 -    );
32 -
33 -    return payload.exp || null;
34 -  } catch {
35 -    return null;
36 -  }
37  +export const createChatWebSocket = ({ conversationId, ...callbacks }) => {
38  +  if (!conversationId) return null;
39  +  return createAuthenticatedWebSocket({
40  +    ...callbacks,
41  +    destination: `/topic/conversations/${conversationId}`,
42  +    publishDestination: `/app/chat/${conversationId}`,
43  +  });
44  };
45 -
46 -const isTokenExpiringSoon = (token) => {
47 -  const expiration = getTokenExpiration(token);
48 -
49 -  if (!expiration) {
50 -    return false;
51 -  }
52 -
53 -  const currentTime = Math.floor(Date.now() / 1000);
54 -
55 -  return expiration - currentTime <= TOKEN_REFRESH_BUFFER_SECONDS;
56 -};
57 -
58 -// =====
59 -// GET VALID ACCESS TOKEN
60 -// =====
61 -
62 -const getValidAccessToken = async () => {
63 -  let accessToken = tokenStorage.getAccessToken();
64 -
65 -  if (!accessToken) {
66 -    return null;
67 -  }
68 -
69 -  if (isTokenExpiringSoon(accessToken)) {
70 -    try {

```

```

71 -     accessToken = await refreshAccessToken();
72 -   } catch (error) {
73 -     console.error(
74 -       "Unable to refresh access token for chat WebSocket:",
75 -       error,
76 -     );
77 -
78 -     return null;
79 -   }
80 - }
81 -
82 - return accessToken;
83 -});
84 -
85 -// =====
86 -// CREATE CHAT WEBSOCKET
87 -// =====
88 -
89 -export const createChatWebSocket = ({
90 -  conversationId,
91 -  onMessage,
92 -  onConnect,
93 -  onDisconnect,
94 -  onError,
95 -}) => {
96 -  if (!conversationId) {
97 -    return null;
98 -  }
99 -
100 -  let manuallyDisconnected = false;
101 -  let subscription = null;
102 -
103 -  const destination = `/topic/conversations/${conversationId}`;
104 -
105 -  const client = new Client({
106 -    brokerURL: WS_URL,
107 -
108 -    // -----
109 -    // AUTHENTICATION
110 -    // -----
111 -
112 -    beforeConnect: async () => {
113 -      const accessToken = await getValidAccessToken();
114 -
115 -      if (!accessToken) {
116 -        throw new Error("No valid access token available for chat WebSocket");
117 -      }
118 -
119 -      client.connectHeaders = {
120 -        Authorization: `Bearer ${accessToken}`,
121 -      };
122 -    },
123 -
124 -    reconnectDelay: RECONNECT_DELAY,
125 -
126 -    heartbeatIncoming: 10000,
127 -
128 -    heartbeatOutgoing: 10000,
129 -
130 -    debug: () => {
131 -      // STOMP debug logging disabled.
132 -    },
133 -
134 -    // -----
135 -    // CONNECT
136 -    // -----
137 -
138 -    onConnect: () => {
139 -      if (manuallyDisconnected) {
140 -        return;

```

```

141 -     }
142 -
143 -     if (subscription) {
144 -         try {
145 -             subscription.unsubscribe();
146 -         } catch (error) {
147 -             console.error("Failed to remove previous chat subscription:", error);
148 -         }
149 -
150 -         subscription = null;
151 -     }
152 -
153 -     subscription = client.subscribe(destination, (message) => {
154 -         try {
155 -             const parsedMessage = JSON.parse(message.body);
156 -
157 -             if (!parsedMessage) {
158 -                 return;
159 -             }
160 -
161 -             onMessage?.(parsedMessage);
162 -         } catch (error) {
163 -             console.error("Failed to parse chat WebSocket message:", error);
164 -
165 -             onError?.(error);
166 -         }
167 -     });
168 -
169 -     onConnect?.();
170 - },
171 -
172 - // -----
173 - // STOMP ERROR
174 - // -----
175 -
176 - onStompError: async (frame) => {
177 -     const errorMessage =
178 -         frame.headers?.message || "Unknown chat broker error";
179 -
180 -     console.error("Chat STOMP broker error:", errorMessage);
181 -
182 -     const message = errorMessage.toLowerCase();
183 -
184 -     if (
185 -         message.includes("invalid access token") ||
186 -         message.includes("authentication") ||
187 -         message.includes("unauthorized") ||
188 -         message.includes("websocket authentication")
189 -     ) {
190 -         try {
191 -             await refreshAccessToken();
192 -
193 -             if (!manuallyDisconnected && !client.active) {
194 -                 client.activate();
195 -             }
196 -         } catch (refreshError) {
197 -             console.error(
198 -                 "Chat WebSocket authentication refresh failed:",
199 -                 refreshError,
200 -             );
201 -
202 -             manuallyDisconnected = true;
203 -
204 -             try {
205 -                 await client.deactivate();
206 -             } catch (deactivateError) {
207 -                 console.error(
208 -                     "Failed to deactivate chat WebSocket:",
209 -                     deactivateError,
210 -                 );

```

```

211 -     }
212 -
213 -     onError?.(refreshError);
214 -   }
215 -
216 -   return;
217 - }
218 -
219 -   onError?.(new Error(errorMessage));
220 - },
221 -
222 -   // -----
223 -   // WEBSOCKET ERROR
224 -   // -----
225 -
226 -   onWebSocketError: (error) => {
227 -     console.error("Chat WebSocket error:", error);
228 -
229 -     onError?.(error);
230 -   },
231 -
232 -   // -----
233 -   // WEBSOCKET CLOSE
234 -   // -----
235 -
236 -   onWebSocketClose: () => {
237 -     subscription = null;
238 -
239 -     onDisconnect?.();
240 -   },
241 -
242 -   // -----
243 -   // STOMP DISCONNECT
244 -   // -----
245 -
246 -   onDisconnect: () => {
247 -     subscription = null;
248 -
249 -     onDisconnect?.();
250 -   },
251 - });
252 -
253 - // =====
254 - // AUTH CLEARED
255 - // =====
256 -
257 - const handleAuthCleared = () => {
258 -   manuallyDisconnected = true;
259 -
260 -   subscription = null;
261 -
262 -   if (client.active) {
263 -     client.deactivate().catch((error) => {
264 -       console.error(
265 -         "Failed to disconnect chat WebSocket after logout:",
266 -         error,
267 -       );
268 -     });
269 -   }
270 - };
271 -
272 - window.addEventListener(AUTH_CLEARED_EVENT, handleAuthCleared);
273 -
274 - // =====
275 - // PUBLIC API
276 - // =====
277 -
278 - return {
279 -   connect: () => {
280 -     manuallyDisconnected = false;

```

```
281 -
282 -     if (!client.active) {
283 -         client.activate();
284 -     }
285 - },
286 -
287 -     disconnect: async () => {
288 -         manuallyDisconnected = true;
289 -
290 -         subscription = null;
291 -
292 -         window.removeEventListener(AUTH_CLEARED_EVENT, handleAuthCleared);
293 -
294 -         if (client.active) {
295 -             await client.deactivate();
296 -         }
297 -     },
298 -
299 -     sendMessage: (content) => {
300 -         if (!client.connected) {
301 -             throw new Error("Chat WebSocket is not connected");
302 -         }
303 -
304 -         client.publish({
305 -             destination: `/app/chat/${conversationId}`,
306 -
307 -             body: JSON.stringify({
308 -                 content,
309 -             }),
310 -         });
311 -     },
312 -
313 -     isConnected: () => {
314 -         return client.connected;
315 -     },
316 - };
317 -};
```

src/services/conversationUpdateWebSocketService.js

FULL CURRENT CONTENT

```
1 import { createAuthenticatedWebSocket } from "../authenticatedWebSocket";
2
3 export const createConversationUpdateWebSocket = ({ onUpdate, ...callbacks }) =>
4   createAuthenticatedWebSocket({
5     ...callbacks,
6     onMessage: onUpdate,
7     destination: "/user/queue/conversation-updates",
8   });
```

src/services/conversationUpdateWebSocketService.js

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/services/conversationUpdateWebSocketService.js
2  +++ after/src/services/conversationUpdateWebSocketService.js
3  @@ -1,307 +1,8 @@
4  -import { Client } from "@stomp/stompjs";
5  +import { createAuthenticatedWebSocket } from "../authenticatedWebSocket";
6
7  -import { refreshAccessToken } from "../api/axiosClient";
8  -
9  -import { tokenStorage, AUTH_CLEARED_EVENT } from "../utils/tokenStorage";
10 -
11 -const WS_URL = import.meta.env.VITE_WS_URL;
12 -
13 -const RECONNECT_DELAY = 5000;
14 -
15 -const TOKEN_REFRESH_BUFFER_SECONDS = 30;
16 -
17 -const CONVERSATION_UPDATE_DESTINATION = "/user/queue/conversation-updates";
18 -
19 -// =====
20 -// JWT EXPIRATION
21 -// =====
22 -
23 -const getTokenExpiration = (token) => {
24 -  try {
25 -    const parts = token.split(".");
26 -
27 -    if (parts.length !== 3) {
28 -      return null;
29 -    }
30 -
31 -    const payload = JSON.parse(
32 -      atob(parts[1].replace(/-/g, "+").replace(/_/g, "/"))
33 -    );
34 -
35 -    return payload.exp || null;
36 -  } catch {
37 -    return null;
38 -  }
39 -};
40 -
41 -const isTokenExpiringSoon = (token) => {
42 -  const expiration = getTokenExpiration(token);
43 -
44 -  if (!expiration) {
45 -    return false;
46 -  }
47 -
48 -  const currentTime = Math.floor(Date.now() / 1000);
49 -
50 -  return expiration - currentTime <= TOKEN_REFRESH_BUFFER_SECONDS;
51 -};
52 -
53 -// =====
54 -// GET VALID ACCESS TOKEN
55 -// =====
56 -
57 -const getValidAccessToken = async () => {
58 -  let accessToken = tokenStorage.getAccessToken();
59 -
60 -  if (!accessToken) {
61 -    return null;
62 -  }
63 -
64 -  if (isTokenExpiringSoon(accessToken)) {
65 -    try {
66 -      accessToken = await refreshAccessToken();
67 -    } catch (error) {
68 -      console.error(
69 -        "Unable to refresh access token for conversation update WebSocket:",
70 -        error,

```

```

71 -     );
72 -
73 -     return null;
74 - }
75 - }
76 -
77 - return accessToken;
78 -});
79 -
80 -// =====
81 -// CREATE CONVERSATION UPDATE WEBSOCKET
82 -// =====
83 -
84 -export const createConversationUpdateWebSocket = ({
85 -  onUpdate,
86 -  onConnect,
87 -  onDisconnect,
88 -  onError,
89 -}) => {
90 -  let manuallyDisconnected = false;
91 -
92 -  let subscription = null;
93 -
94 -  const client = new Client({
95 -    brokerURL: WS_URL,
96 -
97 -    // -----
98 -    // AUTHENTICATION
99 -    // -----
100 -
101 -    beforeConnect: async () => {
102 -      const accessToken = await getValidAccessToken();
103 -
104 -      if (!accessToken) {
105 -        throw new Error(
106 -          "No valid access token available for conversation update WebSocket",
107 -        );
108 -      }
109 -
110 -      client.connectHeaders = {
111 -        Authorization: `Bearer ${accessToken}`,
112 -      };
113 -    },
114 -
115 -    reconnectDelay: RECONNECT_DELAY,
116 -
117 -    heartbeatIncoming: 10000,
118 -
119 -    heartbeatOutgoing: 10000,
120 -
121 -    debug: () => {
122 -      // STOMP debug logging disabled.
123 -    },
124 -
125 -    // -----
126 -    // CONNECT
127 -    // -----
128 -
129 -    onConnect: () => {
130 -      if (manuallyDisconnected) {
131 -        return;
132 -      }
133 -
134 -      /*
135 -       * Prevent duplicate subscriptions
136 -       * after reconnect.
137 -       */
138 -      if (subscription) {
139 -        try {
140 -          subscription.unsubscribe();

```

```

141 -     } catch (error) {
142 -         console.error(
143 -             "Failed to remove previous conversation update subscription:",
144 -             error,
145 -         );
146 -     }
147 -
148 -     subscription = null;
149 - }
150 -
151 - subscription = client.subscribe(
152 -     CONVERSATION_UPDATE_DESTINATION,
153 -     (message) => {
154 -         try {
155 -             const update = JSON.parse(message.body);
156 -
157 -             if (!update) {
158 -                 return;
159 -             }
160 -
161 -             onUpdate?.(update);
162 -         } catch (error) {
163 -             console.error(
164 -                 "Failed to parse conversation update WebSocket message:",
165 -                 error,
166 -             );
167 -
168 -             onError?.(error);
169 -         }
170 -     },
171 - );
172 -
173 - onConnect?.();
174 - },
175 -
176 - // -----
177 - // STOMP ERROR
178 - // -----
179 -
180 - onStompError: async (frame) => {
181 -     const errorMessage =
182 -         frame.headers?.message || "Unknown conversation update broker error";
183 -
184 -     console.error("Conversation update STOMP broker error:", errorMessage);
185 -
186 -     const message = errorMessage.toLowerCase();
187 -
188 -     if (
189 -         message.includes("invalid access token") ||
190 -         message.includes("authentication") ||
191 -         message.includes("unauthorized") ||
192 -         message.includes("websocket authentication")
193 -     ) {
194 -         try {
195 -             await refreshAccessToken();
196 -
197 -             if (!manuallyDisconnected && !client.active) {
198 -                 client.activate();
199 -             }
200 -         } catch (refreshError) {
201 -             console.error(
202 -                 "Conversation update WebSocket authentication refresh failed:",
203 -                 refreshError,
204 -             );
205 -
206 -             manuallyDisconnected = true;
207 -
208 -             try {
209 -                 await client.deactivate();
210 -             } catch (deactivateError) {

```

```

211 -         console.error(
212 -             "Failed to deactivate conversation update WebSocket:",
213 -             deactivateError,
214 -         );
215 -     }
216 -
217 -     onError?.(refreshError);
218 - }
219 -
220 -     return;
221 - }
222 -
223 -     onError?.(new Error(errorMessage));
224 - },
225 -
226 -     // -----
227 -     // WEBSOCKET ERROR
228 -     // -----
229 -
230 -     onWebSocketError: (error) => {
231 -         console.error("Conversation update WebSocket error:", error);
232 -
233 -         onError?.(error);
234 -     },
235 -
236 -     // -----
237 -     // WEBSOCKET CLOSE
238 -     // -----
239 -
240 -     onWebSocketClose: () => {
241 -         subscription = null;
242 -
243 -         onDisconnect?.();
244 -     },
245 -
246 -     // -----
247 -     // STOMP DISCONNECT
248 -     // -----
249 -
250 -     onDisconnect: () => {
251 -         subscription = null;
252 -
253 -         onDisconnect?.();
254 -     },
255 +export const createConversationUpdateWebSocket = ({ onUpdate, ...callbacks }) =>
256 +createAuthenticatedWebSocket({
257 +    ...callbacks,
258 +    onMessage: onUpdate,
259 +    destination: "/user/queue/conversation-updates",
260 });
261 -
262 -     // =====
263 -     // AUTH CLEARED
264 -     // =====
265 -
266 -     const handleAuthCleared = () => {
267 -         manuallyDisconnected = true;
268 -
269 -         subscription = null;
270 -
271 -         if (client.active) {
272 -             client.deactivate().catch((error) => {
273 -                 console.error(
274 -                     "Failed to disconnect conversation update WebSocket after logout:",
275 -                     error,
276 -                 );
277 -             });
278 -         }
279 -     };
280 -

```

```
281 - window.addEventListener(AUTH_CLEARED_EVENT, handleAuthCleared);
282 -
283 - // =====
284 - // PUBLIC API
285 - // =====
286 -
287 - return {
288 -   connect: () => {
289 -     manuallyDisconnected = false;
290 -
291 -     if (!client.active) {
292 -       client.activate();
293 -     }
294 -   },
295 -
296 -   disconnect: async () => {
297 -     manuallyDisconnected = true;
298 -
299 -     subscription = null;
300 -
301 -     window.removeEventListener(AUTH_CLEARED_EVENT, handleAuthCleared);
302 -
303 -     if (client.active) {
304 -       await client.deactivate();
305 -     }
306 -   },
307 -
308 -   isActive: () => {
309 -     return client.active;
310 -   },
311 -
312 -   isConnected: () => {
313 -     return client.connected;
314 -   },
315 - };
316 -};
```

```

1  /** Shared lifecycle; each feature retains its own independently owned connection. */
2  export function createStompConnection({ createClient, getToken, refreshToken,
3    authEvents, authClearedEvent, destination, onMessage, onConnect, onDisconnect,
4    onError, publishDestination, brokerURL }) {
5    let running = false;
6    let disposed = false;
7    let generation = 0;
8    let subscription = null;
9    let connected = false;
10   let refreshRequired = false;
11   let authenticationRetried = false;
12   let listening = false;
13   const report = (error) => { if (running) onError?.(error); };
14   const notifyClosed = () => {
15     subscription = null;
16     if (connected) { connected = false; if (running) onDisconnect?.(); }
17   };
18   const stop = () => {
19     running = false;
20     generation++;
21     connected = false;
22     subscription = null;
23     if (listening) authEvents.removeEventListener(authClearedEvent, stop);
24     listening = false;
25     return client.deactivate().catch(() => {});
26   };
27   const client = createClient({
28     brokerURL, reconnectDelay: 5000, connectionTimeout: 10000,
29     heartbeatIncoming: 10000, heartbeatOutgoing: 10000, debug: () => {},
30     beforeConnect: async () => {
31       const attempt = generation;
32       try {
33         let token = getToken();
34         if (!running || !token) { await stop(); return; }
35         if (refreshRequired || tokenExpiringSoon(token)) {
36           refreshRequired = false;
37           token = await refreshToken();
38         }
39         // Logout/disposal while refresh was pending must never reopen a socket.
40         if (!running || attempt !== generation) return;
41         if (!token || token !== getToken()) { await stop(); return; }
42         client.connectHeaders = { Authorization: `Bearer ${token}` };
43       } catch {
44         // beforeConnect must settle normally: STOMP does not catch its rejection.
45         report(new Error("Unable to authenticate live updates. Please sign in again."));
46         await stop();
47       }
48     },
49     onConnect: () => {
50       if (!running) return;
51       subscription?.unsubscribe();
52       subscription = client.subscribe(destination, frame => {
53         if (!running) return;
54         try {
55           const value = JSON.parse(frame.body);
56           if (value && typeof value === "object" && !Array.isArray(value)) onMessage?.(value);
57         } catch { report(new Error("Unable to read a live update.")); }
58       });
59       connected = true;
60       authenticationRetried = false;
61       onConnect?.();
62     },
63     onStompError: frame => {
64       if (!running) return;
65       const authError = /authentication|unauthorized|invalid access token/i.test(frame.headers?.message || "");
66       if (authError && !authenticationRetried) {
67         authenticationRetried = true;
68         refreshRequired = true;
69       } else {
70         report(new Error(authError ? "Live authentication was rejected. Please sign in again." : "Live subscrip

```

```

> on was rejected."));
71     void stop();
72     onDisconnect?.();
73     return;
74 }
75 // Reconnect runs beforeConnect and uses the latest token, even if ERROR stays open.
76 notifyClosed();
77 client.forceDisconnect();
78 },
79 onWebSocketError: () => report(new Error("Live connection is unavailable.")),
80 onWebSocketClose: notifyClosed,
81 onDisconnect: notifyClosed,
82 });
83 return {
84   connect() {
85     if (disposed || running) return;
86     running = true;
87     generation++;
88     authEvents.addEventListener(authClearedEvent, stop);
89     listening = true;
90     client.activate();
91   },
92   disconnect() { disposed = true; return stop(); },
93   isActive: () => running && client.active,
94   isConnected: () => running && connected && client.connected,
95   getClient: () => client,
96   sendMessage(content) {
97     if (!running || !connected || !client.connected || !publishDestination) throw new Error("Chat WebSocket is
> not connected");
98     client.publish({ destination: publishDestination, body: JSON.stringify({ content }) });
99   },
100 };
101 }
102
103 export function tokenExpiringSoon(token, now = Date.now()) {
104   try {
105     const payload = JSON.parse(atob(token.split(".")[1].replace(/-/g, "+").replace(/_/g, "/")));
106     return typeof payload.exp === "number" && payload.exp - Math.floor(now / 1000) <= 30;
107   } catch { return false; } // Backend remains responsible for token validation.
108 }

```

```

1  --- /dev/null
2  +++ after/src/services/stompConnection.js
3  @@ -0,0 +1,108 @@
4  +/** Shared lifecycle; each feature retains its own independently owned connection. */
5  +export function createStompConnection({ createClient, getToken, refreshToken,
6  +  authEvents, authClearedEvent, destination, onMessage, onConnect, onDisconnect,
7  +  onError, publishDestination, brokerURL }) {
8  +  let running = false;
9  +  let disposed = false;
10 +  let generation = 0;
11 +  let subscription = null;
12 +  let connected = false;
13 +  let refreshRequired = false;
14 +  let authenticationRetried = false;
15 +  let listening = false;
16 +  const report = (error) => { if (running) onError?.(error); };
17 +  const notifyClosed = () => {
18 +    subscription = null;
19 +    if (connected) { connected = false; if (running) onDisconnect?.(); }
20 +  };
21 +  const stop = () => {
22 +    running = false;
23 +    generation++;
24 +    connected = false;
25 +    subscription = null;
26 +    if (listening) authEvents.removeEventListener(authClearedEvent, stop);
27 +    listening = false;
28 +    return client.deactivate().catch(() => {});
29 +  };
30 +  const client = createClient({
31 +    brokerURL, reconnectDelay: 5000, connectionTimeout: 10000,
32 +    heartbeatIncoming: 10000, heartbeatOutgoing: 10000, debug: () => {},
33 +    beforeConnect: async () => {
34 +      const attempt = generation;
35 +      try {
36 +        let token = getToken();
37 +        if (!running || !token) { await stop(); return; }
38 +        if (refreshRequired || tokenExpiringSoon(token)) {
39 +          refreshRequired = false;
40 +          token = await refreshToken();
41 +        }
42 +        // Logout/disposal while refresh was pending must never reopen a socket.
43 +        if (!running || attempt !== generation) return;
44 +        if (!token || token !== getToken()) { await stop(); return; }
45 +        client.connectHeaders = { Authorization: `Bearer ${token}` };
46 +      } catch {
47 +        // beforeConnect must settle normally: STOMP does not catch its rejection.
48 +        report(new Error("Unable to authenticate live updates. Please sign in again."));
49 +        await stop();
50 +      }
51 +    },
52 +    onConnect: () => {
53 +      if (!running) return;
54 +      subscription?.unsubscribe();
55 +      subscription = client.subscribe(destination, frame => {
56 +        if (!running) return;
57 +        try {
58 +          const value = JSON.parse(frame.body);
59 +          if (value && typeof value === "object" && !Array.isArray(value)) onMessage?.(value);
60 +        } catch { report(new Error("Unable to read a live update.")); }
61 +      });
62 +      connected = true;
63 +      authenticationRetried = false;
64 +      onConnect?.();
65 +    },
66 +    onStompError: frame => {
67 +      if (!running) return;
68 +      const authError = /authentication|unauthorized|invalid access token/i.test(frame.headers?.message || "");
69 +      if (authError && !authenticationRetried) {
70 +        authenticationRetried = true;

```

```

71 +     refreshRequired = true;
72 +   } else {
73 +     report(new Error(authError ? "Live authentication was rejected. Please sign in again." : "Live subscript
> ion was rejected."));
74 +     void stop();
75 +     onDisconnect?.();
76 +     return;
77 +   }
78 +   // Reconnect runs beforeConnect and uses the latest token, even if ERROR stays open.
79 +   notifyClosed();
80 +   client.forceDisconnect();
81 + },
82 + onWebSocketError: () => report(new Error("Live connection is unavailable.")),
83 + onWebSocketClose: notifyClosed,
84 + onDisconnect: notifyClosed,
85 + });
86 + return {
87 +   connect() {
88 +     if (disposed || running) return;
89 +     running = true;
90 +     generation++;
91 +     authEvents.addEventListener(authClearedEvent, stop);
92 +     listening = true;
93 +     client.activate();
94 +   },
95 +   disconnect() { disposed = true; return stop(); },
96 +   isActive: () => running && client.active,
97 +   isConnected: () => running && connected && client.connected,
98 +   getClient: () => client,
99 +   sendMessage(content) {
100 +     if (!running || !connected || !client.connected || !publishDestination) throw new Error("Chat WebSocket is
> not connected");
101 +     client.publish({ destination: publishDestination, body: JSON.stringify({ content }) });
102 +   },
103 + };
104 +}
105 +
106 +export function tokenExpiringSoon(token, now = Date.now()) {
107 +  try {
108 +    const payload = JSON.parse(atob(token.split(".")[1].replace(/-/g, "+").replace(/_/g, "/")));
109 +    return typeof payload.exp === "number" && payload.exp - Math.floor(now / 1000) <= 30;
110 +  } catch { return false; } // Backend remains responsible for token validation.
111 +}

```

src/services/webSocketService.js

FULL CURRENT CONTENT

```
1 import { createAuthenticatedWebSocket } from "../authenticatedWebSocket";
2
3 export const createNotificationWebSocket = ({ onNotification, ...callbacks }) =>
4   createAuthenticatedWebSocket({
5     ...callbacks,
6     onMessage: onNotification,
7     destination: "/user/queue/notifications",
8   });
```

```

1  --- before/src/services/webSocketService.js
2  +++ after/src/services/webSocketService.js
3  @@ -1,303 +1,8 @@
4  -import { Client } from "@stomp/stompjs";
5  +import { createAuthenticatedWebSocket } from "../authenticatedWebSocket";
6
7  -import { refreshAccessToken } from "../api/axiosClient";
8  -
9  -import { tokenStorage, AUTH_CLEARED_EVENT } from "../utils/tokenStorage";
10 -
11 -const WS_URL = import.meta.env.VITE_WS_URL;
12 -
13 -const NOTIFICATION_DESTINATION = "/user/queue/notifications";
14 -
15 -const RECONNECT_DELAY = 5000;
16 -
17 -const TOKEN_REFRESH_BUFFER_SECONDS = 30;
18 -
19 -// =====
20 -// JWT EXPIRATION
21 -// =====
22 -
23 -const getTokenExpiration = (token) => {
24 -  try {
25 -    const parts = token.split(".");
26 -
27 -    if (parts.length !== 3) {
28 -      return null;
29 -    }
30 -
31 -    const payload = JSON.parse(
32 -      atob(parts[1].replace(/-/g, "+").replace(/_/g, "/"))
33 -    );
34 -
35 -    return payload.exp || null;
36 -  } catch {
37 -    return null;
38 -  }
39 -};
40 -
41 -const isTokenExpiringSoon = (token) => {
42 -  const expiration = getTokenExpiration(token);
43 -
44 -  if (!expiration) {
45 -    /*
46 -     * If the token cannot be decoded,
47 -     * let the backend perform validation.
48 -     */
49 -    return false;
50 -  }
51 -
52 -  const currentTime = Math.floor(Date.now() / 1000);
53 -
54 -  return expiration - currentTime <= TOKEN_REFRESH_BUFFER_SECONDS;
55 -};
56 -
57 -// =====
58 -// GET VALID ACCESS TOKEN
59 -// =====
60 -
61 -const getValidAccessToken = async () => {
62 -  let accessToken = tokenStorage.getAccessToken();
63 -
64 -  if (!accessToken) {
65 -    return null;
66 -  }
67 -
68 -  if (isTokenExpiringSoon(accessToken)) {
69 -    try {
70 -      accessToken = await refreshAccessToken();

```

```

71 -     } catch (error) {
72 -         console.error("Unable to refresh access token for WebSocket:", error);
73 -
74 -         return null;
75 -     }
76 - }
77 -
78 - return accessToken;
79 -});
80 -
81 -// =====
82 -// CREATE NOTIFICATION WEBSOCKET
83 -// =====
84 -
85 -export const createNotificationWebSocket = ({
86 -    onNotification,
87 -    onConnect,
88 -    onDisconnect,
89 -    onError,
90 -}) => {
91 -    let manuallyDisconnected = false;
92 -
93 -    let subscription = null;
94 -
95 -    const client = new Client({
96 -        brokerURL: WS_URL,
97 -
98 -        /*
99 -         * Get the latest access token before every
100 -        * connection/reconnection.
101 -        */
102 -        beforeConnect: async () => {
103 -            const accessToken = await getValidAccessToken();
104 -
105 -            if (!accessToken) {
106 -                throw new Error("No valid access token available for WebSocket");
107 -            }
108 -
109 -            client.connectHeaders = {
110 -                Authorization: `Bearer ${accessToken}`,
111 -            };
112 -        },
113 -
114 -        reconnectDelay: RECONNECT_DELAY,
115 -
116 -        heartbeatIncoming: 10000,
117 -
118 -        heartbeatOutgoing: 10000,
119 -
120 -        debug: () => {
121 -            /*
122 -             * STOMP debug logging disabled normally.
123 -            */
124 -        },
125 -
126 -        // =====
127 -        // CONNECT
128 -        // =====
129 -
130 -        onConnect: () => {
131 -            if (manuallyDisconnected) {
132 -                return;
133 -            }
134 -
135 -            /*
136 -             * Prevent duplicate subscriptions if STOMP
137 -             * reconnects using the same client.
138 -            */
139 -            if (subscription) {
140 -                try {

```

```

141 -     subscription.unsubscribe();
142 -   } catch (error) {
143 -     console.error(
144 -       "Failed to remove old notification subscription:",
145 -       error,
146 -     );
147 -   }
148 -
149 -   subscription = null;
150 - }
151 -
152 - subscription = client.subscribe(NOTIFICATION_DESTINATION, (message) => {
153 -   try {
154 -     const notification = JSON.parse(message.body);
155 -
156 -     if (!notification) {
157 -       console.warn("Notification WS: empty notification");
158 -
159 -       return;
160 -     }
161 -
162 -     onNotification?.(notification);
163 -   } catch (error) {
164 -     console.error("Failed to parse WebSocket notification:", error);
165 -
166 -     onError?.(error);
167 -   }
168 - });
169 -
170 - onConnect?.();
171 - },
172 -
173 - // =====
174 - // STOMP ERROR
175 - // =====
176 -
177 - onStompError: async (frame) => {
178 -   const errorMessage = frame.headers?.message || "Unknown broker error";
179 -
180 -   console.error("STOMP broker error:", errorMessage);
181 -
182 -   const message = errorMessage.toLowerCase();
183 -
184 -   /*
185 -    * Authentication failure.
186 -    *
187 -    * Refresh the token and allow the STOMP
188 -    * client to reconnect using beforeConnect().
189 -    */
190 -   if (
191 -     message.includes("invalid access token") ||
192 -     message.includes("authentication") ||
193 -     message.includes("unauthorized") ||
194 -     message.includes("websocket authentication")
195 -   ) {
196 -     try {
197 -       await refreshAccessToken();
198 -
199 -       if (!manuallyDisconnected && !client.active) {
200 -         client.activate();
201 -       }
202 -     } catch (refreshError) {
203 -       console.error(
204 -         "WebSocket authentication refresh failed:",
205 -         refreshError,
206 -       );
207 -
208 -       manuallyDisconnected = true;
209 -
210 -       try {

```

```

211 -         await client.deactivate();
212 -     } catch (deactivateError) {
213 -         console.error("Failed to deactivate WebSocket:", deactivateError);
214 -     }
215 -
216 -     onError?.(refreshError);
217 - }
218 -
219 -     return;
220 - }
221 -
222 -     onError?.(new Error(errorMessage));
223 - },
224 -
225 -     // =====
226 -     // WEBSOCKET ERROR
227 -     // =====
228 -
229 -     onWebSocketError: (error) => {
230 -         console.error("WebSocket error:", error);
231 -
232 -         onError?.(error);
233 -     },
234 -
235 -     // =====
236 -     // WEBSOCKET CLOSE
237 -     // =====
238 -
239 -     onWebSocketClose: () => {
240 -         subscription = null;
241 -
242 -         onDisconnect?.();
243 -     },
244 -
245 -     // =====
246 -     // STOMP DISCONNECT
247 -     // =====
248 -
249 -     onDisconnect: () => {
250 -         subscription = null;
251 -
252 -         onDisconnect?.();
253 -     },
254 + export const createNotificationWebSocket = ({ onNotification, ...callbacks }) =>
255 +   createAuthenticatedWebSocket({
256 +     ...callbacks,
257 +     onMessage: onNotification,
258 +     destination: "/user/queue/notifications",
259   });
260 -
261 -     // =====
262 -     // AUTH CLEARED
263 -     // =====
264 -
265 -     const handleAuthCleared = () => {
266 -         manuallyDisconnected = true;
267 -
268 -         subscription = null;
269 -
270 -         if (client.active) {
271 -             client.deactivate().catch((error) => {
272 -                 console.error("Failed to disconnect WebSocket after logout:", error);
273 -             });
274 -         }
275 -     };
276 -
277 -     window.addEventListener(AUTH_CLEARED_EVENT, handleAuthCleared);
278 -
279 -     // =====
280 -     // PUBLIC API

```

```
281 - // =====
282 -
283 - return {
284 -   connect: () => {
285 -     manuallyDisconnected = false;
286 -
287 -     if (!client.active) {
288 -       client.activate();
289 -     }
290 -   },
291 -
292 -   disconnect: async () => {
293 -     manuallyDisconnected = true;
294 -
295 -     subscription = null;
296 -
297 -     window.removeEventListener(AUTH_CLEARED_EVENT, handleAuthCleared);
298 -
299 -     if (client.active) {
300 -       await client.deactivate();
301 -     }
302 -   },
303 -
304 -   isActive: () => {
305 -     return client.active;
306 -   },
307 -
308 -   getClient: () => {
309 -     return client;
310 -   },
311 - };
312 -};
```

```

1 import { createServer } from "../node_modules/vite/dist/node/index.js";
2 import { fileURLToPath } from "node:url";
3 import assert from "node:assert/strict";
4 const { chromium } = await import(process.env.PLAYWRIGHT_MODULE || "playwright");
5 const origin = "http://127.0.0.1:41816";
6 process.env.VITE_API_BASE_URL = origin;
7 process.env.VITE_WS_URL = origin.replace("http", "ws") + "/ws";
8 const server = await createServer({ root: fileURLToPath(new URL("../", import.meta.url)),
9   server: { host: "127.0.0.1", port: 41816, strictPort: true } });
10 let browser;
11 try {
12   await server.listen(); browser = await chromium.launch({channel:"msedge",headless:true});
13   const page = await browser.newPage(), errors = [], destinations = [];
14   page.on("pageerror", e => errors.push(e.message));
15   let pendingRoute;
16   await page.route("**/*", route => {
17     const url=new URL(route.request().url());
18     assert.equal(url.origin,origin,"External traffic is prohibited");
19     if(url.pathname==="/fixture") return route.fulfill({contentType:"text/html",body:<html><body>Local fixture</
20 > body></html>"});
21     if(url.pathname==="/api/auth/refresh") { pendingRoute=route; return; }
22     return route.continue();
23   });
24   await page.routeWebSocket("**/ws", socket => {
25     socket.onMessage(frame => {
26       const text=String(frame);
27       if(/^(\CONNECT|STOMP)/.test(text)) socket.send("CONNECTED\nversion:1.2\nheart-beat:0,0\n\n0");
28       if(text.startsWith("SUBSCRIBE")) {
29         const destination=text.match(/\\ndestination:([^\n]+)/)[1],id=text.match(/\\nid:([^\n]+)/)[1];
30         destinations.push(destination);
31         socket.send("MESSAGE\nsubscription:${id}\nmessage-id:1\nndestination:${destination}\n\n{id:1}\n0");
32       }
33       if(text.startsWith("DISCONNECT")) socket.close();
34     });
35   });
36   await page.goto(origin+"/fixture");
37   await page.evaluate(async () => {
38     window.storage=(await import("/src/utils/tokenStorage.js")).tokenStorage;
39     window.refresh=(await import("/src/api/axiosClient.js")).refreshAccessToken;
40     window.storage.saveAuth("old-access","old-refresh",{id:1});
41   });
42   for(const scenario of ["logout","new-login-success","new-login-failure"]) {
43     pendingRoute=null;
44     await page.evaluate(() => { window.storage.saveAuth("old-access","old-refresh",{id:1}); window.result=window.
45 > refresh().then(()=>"success",()=>>"rejected"); });
46     for(let i=0;!pendingRoute&&i<200;i++) await new Promise(r=>setTimeout(r,10));
47     assert.ok(pendingRoute);
48     await page.evaluate(mode => mode===>"logout" ? window.storage.clear() : window.storage.saveAuth("new-access",
49 > new-refresh",{id:2}),scenario);
50     await pendingRoute.fulfill({status:scenario.endsWith("failure")?401:200,json:{accessToken:"stale-access",refr
51 > eshToken:"stale-refresh"}});
52     assert.equal(await page.evaluate(()=>window.result),"rejected");
53     assert.equal(await page.evaluate(()=>window.storage.getAccessToken()),scenario===>"logout"?null:"new-access");
54   }
55   await page.evaluate(async () => {
56     const {createNotificationWebSocket}=await import("/src/services/webSocketService.js");
57     const {createConversationUpdateWebSocket}=await import("/src/services/conversationUpdateWebSocketService.js")
58 > ;
59     const {createChatWebSocket}=await import("/src/services/chatWebSocketService.js");
60     window.received=[];
61     window.sockets=[createNotificationWebSocket({onNotification:x=>window.received.push(x)}),
62     createConversationUpdateWebSocket({onUpdate:x=>window.received.push(x)}),
63     createChatWebSocket({conversationId:12,onMessage:x=>window.received.push(x)}});
64     window.sockets.forEach(s=>s.connect());
65   });
66   await page.waitForFunction(()=>window.received.length===3);
67   assert.deepEqual(destinations.sort(),["/topic/conversations/12","/user/queue/conversation-updates","/user/queue
68 > /notifications"].sort());
69   await page.evaluate(()=>window.storage.clear());
70   await page.waitForFunction(()=>window.sockets.every(s=>!s.isActive()));

```

tests/liveAuth.browser.mjs

FULL CURRENT CONTENT

```
65   assert.deepEqual(errors, []);
66   console.log("PASS: stale refresh cannot restore logout or overwrite/clear newer login; all three adapters subsc
  > ribe and deliver; logout stops all clients.");
67 } finally { await browser?.close(); await server.close(); }
```

```

1 --- /dev/null
2 +++ after/tests/liveAuth.browser.mjs
3 @@ -0,0 +1,67 @@
4 +import { createServer } from "../node_modules/vite/dist/node/index.js";
5 +import { fileURLToPath } from "node:url";
6 +import assert from "node:assert/strict";
7 +const { chromium } = await import(process.env.PLAYWRIGHT_MODULE || "playwright");
8 +const origin = "http://127.0.0.1:41816";
9 +process.env.VITE_API_BASE_URL = origin;
10 +process.env.VITE_WS_URL = origin.replace("http", "ws") + "/ws";
11 +const server = await createServer({ root: fileURLToPath(new URL("../", import.meta.url)),
12 +  server: { host: "127.0.0.1", port: 41816, strictPort: true } });
13 +let browser;
14 +try {
15 +  await server.listen(); browser = await chromium.launch({channel:"msedge",headless:true});
16 +  const page = await browser.newPage(), errors = [], destinations = [];
17 +  page.on("pageerror", e => errors.push(e.message));
18 +  let pendingRoute;
19 +  await page.route("**/*", route => {
20 +    const url=new URL(route.request().url());
21 +    assert.equal(url.origin,origin,"External traffic is prohibited");
22 +    if(url.pathname==="/fixture") return route.fulfill({contentType:"text/html",body:"<html><body>Local fixture
23 > /body</html>"});
24 +    if(url.pathname==="/api/auth/refresh") { pendingRoute=route; return; }
25 +    return route.continue();
26 +  });
27 +  await page.routeWebSocket("**/ws", socket => {
28 +    socket.onMessage(frame => {
29 +      const text=String(frame);
30 +      if(/^(CONNECT|STOMP)/.test(text)) socket.send("CONNECTED\nversion:1.2\nheart-beat:0,0\n\n");
31 +      if(text.startsWith("SUBSCRIBE")) {
32 +        const destination=text.match(/&#x2F;destination:([^\n]+)/)[1],id=text.match(/&#x2F;nid:([^\n]+)/)[1];
33 +        destinations.push(destination);
34 +        socket.send("MESSAGE\nsubscription:${id}\nmessage-id:1\ndestination:${destination}\n\n{id:1}\n");
35 +      }
36 +      if(text.startsWith("DISCONNECT")) socket.close();
37 +    });
38 +    await page.goto(origin+"/fixture");
39 +    await page.evaluate(async () => {
40 +      window.storage=(await import("/src/utils/tokenStorage.js")).tokenStorage;
41 +      window.refresh=(await import("/src/api/axiosClient.js")).refreshAccessToken;
42 +      window.storage.saveAuth("old-access","old-refresh",{id:1});
43 +    });
44 +    for(const scenario of ["logout","new-login-success","new-login-failure"]) {
45 +      pendingRoute=null;
46 +      await page.evaluate(() => { window.storage.saveAuth("old-access","old-refresh",{id:1}); window.result=window
47 > .refresh().then(()=>"success",()=>&#x22;rejected"); });
48 +      for(let i=0;!pendingRoute&#x26amp;i<200;i++) await new Promise(r=>setTimeout(r,10));
49 +      assert.ok(pendingRoute);
50 +      await page.evaluate(mode => mode==="logout" ? window.storage.clear() : window.storage.saveAuth("new-access",
51 > "new-refresh",{id:2}),scenario);
52 +      await pendingRoute.fulfill({status:scenario.endsWith("failure")?401:200,json:{accessToken:"stale-access",ref
53 > reshToken:"stale-refresh"}});
54 +      assert.equal(await page.evaluate(()=>window.result),"rejected");
55 +      assert.equal(await page.evaluate(()=>window.storage.getAccessToken()),scenario==="logout"?null:"new-access")
56 > ;
57 +    }
58 +    await page.evaluate(async () => {
59 +      const {createNotificationWebSocket}=await import("/src/services/webSocketService.js");
60 +      const {createConversationUpdateWebSocket}=await import("/src/services/conversationUpdateWebSocketService.js"
61 > );
62 +      const {createChatWebSocket}=await import("/src/services/chatWebSocketService.js");
63 +      window.received=[];
64 +      window.sockets=[createNotificationWebSocket({onNotification:x=>window.received.push(x)}),
65 +        createConversationUpdateWebSocket({onUpdate:x=>window.received.push(x)}),
66 +        createChatWebSocket({conversationId:12,onMessage:x=>window.received.push(x)}})];
67 +      window.sockets.forEach(s=>s.connect());
68 +    });
69 +    await page.waitForFunction(()=>window.received.length===3);

```

tests/liveAuth.browser.mjs

UNIFIED DIFF: + ADDED / - REMOVED

```
65 + assert.deepEqual(destinations.sort(),["/topic/conversations/12","/user/queue/conversation-updates","/user/queue/notifications"].sort());
66 + await page.evaluate(()=>window.storage.clear());
67 + await page.waitForFunction(()=>window.sockets.every(s=>!s.isActive()));
68 + assert.deepEqual(errors,[]);
69 + console.log("PASS: stale refresh cannot restore logout or overwrite/clear newer login; all three adapters subscribe and deliver; logout stops all clients.");
70 +} finally { await browser?.close(); await server.close(); }
```

```

1 import test from "node:test";
2 import assert from "node:assert/strict";
3 import { createStompConnection, tokenExpiringSoon } from "../src/services/stompConnection.js";
4
5 const jwt = exp => `x.${btoa(JSON.stringify({ exp }))).x`;
6 function fixture(refresh) {
7   let token = jwt(9999999999), config, handler;
8   const events = new EventTarget(), errors = [], messages = [];
9   let opened = 0, closed = 0, subscriptions = 0, unsubscribed = 0;
10  const client = { active: false, connected: false, connectHeaders: {},
11    activate() { this.active = true; },
12    async deactivate() { this.active = false; this.connected = false; },
13    forceDisconnect() { this.connected = false; config.onWebSocketClose(); },
14    subscribe(destination, callback) { assert.equal(destination, "/topic/conversations/12"); subscriptions++; han
15  dler = callback; return { unsubscribe() { unsubscribed++; } }; },
16  };
17  const socket = createStompConnection({ createClient: c => { config = c; return client; },
18    destination: "/topic/conversations/12", getToken: () => token,
19    refreshToken: async () => { const value = await refresh?(); if (value) token = value; return value; },
20    authEvents: events, authClearedEvent: "logout", onMessage: m => messages.push(m),
21    onError: e => errors.push(e), onConnect: () => opened++, onDisconnect: () => closed++,
22  });
23  return { socket, client, events, errors, messages, config,
24    setToken: t => { token = t; }, emit: body => handler({ body }),
25    connect: async () => { socket.connect(); await config.beforeConnect(); if (client.active) { client.connected
26  > = true; config.onConnect(); } },
27    counts: () => ({ opened, closed, subscriptions, unsubscribed }) };
28 }
29
30 test("expiry threshold and malformed tokens leave validation to backend", () => {
31   assert.equal(tokenExpiringSoon(jwt(130),100000),true);
32   assert.equal(tokenExpiringSoon(jwt(131),100000),false);
33   assert.equal(tokenExpiringSoon("invalid"),false);
34 });
35
36 test("connect and reconnect use current token and one subscription", async () => {
37   const f=fixture(); await f.connect(); f.emit('{"id":1}');
38   f.setToken(jwt(9999999998)); await f.config.beforeConnect(); f.config.onConnect();
39   assert.equal(f.client.connectHeaders.Authorization,`Bearer ${jwt(9999999998)}`);
40   assert.deepEqual(f.counts(),{opened:2,closed:0,subscriptions:2,unsubscribed:1});
41   assert.deepEqual(f.messages,[{id:1}]); await f.socket.disconnect();
42 });
43
44 test("malformed frame reports safe error and later valid update works", async () => {
45   const f=fixture(); await f.connect(); f.emit("secret invalid payload"); f.emit('{"id":2}');
46   assert.equal(f.errors.length,1); assert.ok(!f.errors[0].message.includes("secret"));
47   assert.deepEqual(f.messages,[{id:2}]); await f.socket.disconnect();
48 });
49
50 test("close callbacks are deduplicated", async () => {
51   const f=fixture(); await f.connect(); f.config.onWebSocketClose(); f.config.onDisconnect();
52   assert.equal(f.counts().closed,1); await f.socket.disconnect();
53 });
54
55 test("logout during refresh stops connection and ignores late frames", async () => {
56   let resolve; const pending=new Promise(r=>{resolve=r;});
57   const f=fixture(()=>pending); await f.connect(); f.setToken(jwt(1));
58   const flight=f.config.beforeConnect(); f.events.dispatchEvent(new Event("logout"));
59   resolve(jwt(9999999999)); await flight; f.emit('{"id":3}');
60   assert.equal(f.client.active,false); assert.equal(f.socket.isConnected(),false);
61   assert.deepEqual(f.messages,[]);
62 });
63
64 test("refresh rejection settles beforeConnect without unhandled rejection", async () => {
65   const f=fixture(()=>Promise.reject(new Error("private"))); f.setToken(jwt(1));
66   f.socket.connect(); await assert.doesNotReject(f.config.beforeConnect());
67   assert.equal(f.client.active,false); assert.equal(f.errors.length,1);
68 });
69
70 test("auth error refreshes on reconnect once and repeated rejection stops", async () => {
71   let refreshes=0; const f=fixture(()=>{refreshes++;return jwt(9999999999)});
72   await f.connect(); f.config.onStompError({headers:{message:"Unauthorized"}});
73   await f.config.beforeConnect(); assert.equal(refreshes,1);
74   f.config.onStompError({headers:{message:"Unauthorized"}});
75   assert.equal(f.client.active,false); assert.equal(f.errors.length,1);
76 });

```

tests/stompConnection.test.js

FULL CURRENT CONTENT

```
69 test("disposal is final and suppresses stale callbacks", async () => {
70   const f=fixture(); await f.connect(); await f.socket.disconnect();
71   f.socket.connect(); f.config.onConnect(); f.emit('{"id":9}'); f.config.onWebSocketError();
72   assert.equal(f.client.active,false); assert.equal(f.counts().opened,1);
73   assert.deepEqual(f.messages,[]); assert.deepEqual(f.errors,[]);
74 });
```

```

1  --- /dev/null
2  +++ after/tests/stompConnection.test.js
3  @@ -0,0 +1,74 @@
4  +import test from "node:test";
5  +import assert from "node:assert/strict";
6  +import { createStompConnection, tokenExpiringSoon } from "../src/services/stompConnection.js";
7  +
8  +const jwt = exp => `x.${btoa(JSON.stringify({ exp }))).x`;
9  +function fixture(refresh) {
10 + let token = jwt(9999999999), config, handler;
11 + const events = new EventTarget(), errors = [], messages = [];
12 + let opened = 0, closed = 0, subscriptions = 0, unsubscribed = 0;
13 + const client = { active: false, connected: false, connectHeaders: {},
14 +   activate() { this.active = true; },
15 +   async deactivate() { this.active = false; this.connected = false; },
16 +   forceDisconnect() { this.connected = false; config.onWebSocketClose(); },
17 +   subscribe(destination, callback) { assert.equal(destination, "/topic/conversations/12"); subscriptions++; ha
18 +   > ndler = callback; return { unsubscribe() { unsubscribed++; } }; },
19 + };
20 + const socket = createStompConnection({ createClient: c => { config = c; return client; },
21 +   destination: "/topic/conversations/12", getToken: () => token,
22 +   refreshToken: async () => { const value = await refresh?(); if (value) token = value; return value; },
23 +   authEvents: events, authClearedEvent: "logout", onMessage: m => messages.push(m),
24 +   onError: e => errors.push(e), onConnect: () => opened++, onDisconnect: () => closed++,
25 + });
26 + return { socket, client, events, errors, messages, config,
27 +   setToken: t => { token = t; }, emit: body => handler({ body }),
28 +   connect: async () => { socket.connect(); await config.beforeConnect(); if (client.active) { client.connected
29 +   > = true; config.onConnect(); } },
30 +   counts: () => ({ opened, closed, subscriptions, unsubscribed }) };
31 +}
32 +
33 +test("expiry threshold and malformed tokens leave validation to backend", () => {
34 + assert.equal(tokenExpiringSoon(jwt(130),100000),true);
35 + assert.equal(tokenExpiringSoon(jwt(131),100000),false);
36 + assert.equal(tokenExpiringSoon("invalid"),false);
37 +});
38 +test("connect and reconnect use current token and one subscription", async () => {
39 + const f=fixture(); await f.connect(); f.emit('{"id":1}');
40 + f.setToken(jwt(9999999998)); await f.config.beforeConnect(); f.config.onConnect();
41 + assert.equal(f.client.connectHeaders.Authorization,`Bearer ${jwt(9999999998)}`);
42 + assert.deepEqual(f.counts(),{opened:2,closed:0,subscriptions:2,unsubscribed:1});
43 + assert.deepEqual(f.messages,[{id:1}]); await f.socket.disconnect();
44 +});
45 +test("malformed frame reports safe error and later valid update works", async () => {
46 + const f=fixture(); await f.connect(); f.emit("secret invalid payload"); f.emit('{"id":2}');
47 + assert.equal(f.errors.length,1); assert.ok(!f.errors[0].message.includes("secret"));
48 + assert.deepEqual(f.messages,[{id:2}]); await f.socket.disconnect();
49 +});
50 +test("close callbacks are deduplicated", async () => {
51 + const f=fixture(); await f.connect(); f.config.onWebSocketClose(); f.config.onDisconnect();
52 + assert.equal(f.counts().closed,1); await f.socket.disconnect();
53 +});
54 +test("logout during refresh stops connection and ignores late frames", async () => {
55 + let resolve; const pending=new Promise(r=>{resolve=r;});
56 + const f=fixture(()=>pending); await f.connect(); f.setToken(jwt(1));
57 + const flight=f.config.beforeConnect(); f.events.dispatchEvent(new Event("logout"));
58 + resolve(jwt(9999999999)); await flight; f.emit('{"id":3}');
59 + assert.equal(f.client.active,false); assert.equal(f.socket.isConnected(),false);
60 + assert.deepEqual(f.messages,[]);
61 +});
62 +test("refresh rejection settles beforeConnect without unhandled rejection", async () => {
63 + const f=fixture(()=>Promise.reject(new Error("private"))); f.setToken(jwt(1));
64 + f.socket.connect(); await assert.doesNotReject(f.config.beforeConnect());
65 + assert.equal(f.client.active,false); assert.equal(f.errors.length,1);
66 +});
67 +test("auth error refreshes on reconnect once and repeated rejection stops", async () => {
68 + let refreshes=0; const f=fixture(()=>{refreshes++;return jwt(9999999999)});
69 + await f.connect(); f.config.onStompError({headers:{message:"Unauthorized"}});
70 + await f.config.beforeConnect(); assert.equal(refreshes,1);

```

tests/stompConnection.test.js

UNIFIED DIFF: + ADDED / - REMOVED

```
69 + f.config.onStompError({headers:{message:"Unauthorized"}});
70 + assert.equal(f.client.active,false); assert.equal(f.errors.length,1);
71 +});
72 +test("disposal is final and suppresses stale callbacks", async () => {
73 + const f=fixture(); await f.connect(); await f.socket.disconnect();
74 + f.socket.connect(); f.config.onConnect(); f.emit('{"id":9}'); f.config.onWebSocketError();
75 + assert.equal(f.client.active,false); assert.equal(f.counts().opened,1);
76 + assert.deepEqual(f.messages,[]); assert.deepEqual(f.errors,[]);
77 +});
```